



Licenciatura em Telecomunicações e Informática

Arquitetura e Tecnologia de Computadores

1st Semester 2024/2025

Rotinas de Atraso e *Timers*

Tiago Gomes

Embedded Systems Research Group (ESRG)
Centro ALGORITMI, Universidade do Minho

mr.gomes@dei.uminho.pt



Rotinas de atraso (delay) - software

- Por software:
 - O tempo de espera (delay) é estabelecido pelo número de ciclos de relógio necessários para executar as instruções associadas à rotina;
 - Podem precisar de pequenos ajustes;
 - Lembrar a rotina `soft_delay_ms()`, ajustada com a ajuda do osciloscópio!

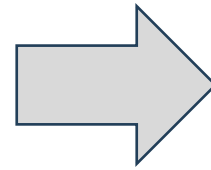
```
/******  
* soft_delay_ms function to generate loop cycles of 1ms *  
******/  
  
void soft_delay_ms(unsigned int ms){  
    unsigned int i, j;  
  
    for (i = 0; i < ms; i++){  
        for(j = 0; j < 2000; j++){  
        }  
    }  
}
```

Rotinas de atraso (delay) - software

```
/*
 * soft_delay_ms function to generate loop cycles of 1ms *
 */

void soft_delay_ms(unsigned int ms){
    unsigned int i, j;

    for (i = 0; i < ms; i++){
        for(j = 0; j < 2000; j++){
        }
    }
}
```



Disassembly

```
38: void soft_delay_ms(unsigned int ms){
39:     unsigned int i, j;
40:
41:     for (i = 0; i < ms; i++){
C:0x020D 8E0F MOV 0x0F,R6
C:0x020F 8F10 MOV 0x10,R7
C:0x0211 751100 MOV 0x11,#0x00
C:0x0214 751200 MOV 0x12,#0x00
C:0x0217 C3 CLR C
C:0x0218 E512 MOV A,0x12
C:0x021A 9510 SUBB A,0x10
C:0x021C E511 MOV A,0x11
C:0x021E 950F SUBB A,0x0F
C:0x0220 5025 JNC C:0247
42:     for(j = 0; j < 2000; j++){
C:0x0222 751300 MOV 0x13,#0x00
C:0x0225 751400 MOV 0x14,#0x00
C:0x0228 C3 CLR C
C:0x0229 E514 MOV A,0x14
C:0x022B 94D0 SUBB A,#PSW(0xD0)
C:0x022D E513 MOV A,0x13
C:0x022F 9407 SUBB A,#0x07
C:0x0231 500A JNC C:023D
43:     }
C:0x0233 0514 INC 0x14
C:0x0235 E514 MOV A,0x14
C:0x0237 7002 JNZ C:023B
C:0x0239 0513 INC 0x13
C:0x023B 80EB SJMP C:0228
44:     }
C:0x023D 0512 INC 0x12
C:0x023F E512 MOV A,0x12
C:0x0241 7002 JNZ C:0245
C:0x0243 0511 INC 0x11
C:0x0245 80D0 SJMP C:0217
45: }
C:0x0247 22 RET
83: void shift_debounce_pbl(void){
```

- Afinal quanto tempo demora!?

Rotinas de atraso (delay) - software

```
/******  
* soft_delay_ms function to generate loop cycles of 1ms *  
******/  
  
void soft_delay_ms(unsigned int ms){  
    unsigned int i, j;  
  
    for (i = 0; i < ms; i++){  
        for(j = 0; j < 2000; j++){  
        }  
    }  
}
```

- **Afinal quanto tempo demora!?**
 - Seria necessário determinar o número de ciclos (NC) máquina necessários à execução de cada instrução para se obter o tempo de execução (na ordem dos us);
- Como obter o tempo de execução de cada ciclo máquina?
 - **Necessário conhecer a frequência de relógio do sistema! (SYSCLK)**

Rotinas de atraso (delay) - software

- Podem ser difíceis de controlar...
 - Como obter exatamente um delay de 35 ms? 500 ms? 100 μ s?
Implementar uma rotina de delay fixo (e.g., 100 μ s, 1 ms) e invocar várias vezes;
- Geralmente a execução do microprocessador fica bloqueada na rotina/função;
- Durante a execução da rotina, outras rotinas de código não são executadas, à excepção de ISRs.

Timers!

Timers/counters (temporizadores)

- Periféricos com hardware dedicado, controlados e configurados através de registos SFR;
- Funcionam de forma independente sem interferir na execução do código;
- Têm tamanho fixo (8 bits, 16 bits, ...);
- Têm associados vários registos (8 bits) e flags (1 bit);
- Flags sinalizam o **overflow** dos registos de contagem do temporizador;
 - Flags são accionadas por hardware, e podem ser testadas/controladas por software;
 - **Flags podem ser usada para gerar uma interrupção para executar uma ISR;**

Timers/counters (temporizadores)

- Periféricos versáteis que permitem vários modos de funcionamento;
- O MCU 8051F388 tem disponíveis vários Timers com vários modos de funcionamento e funções:

Mode	Timer 0 & 1	Timer 2	Timer 3	Timer 4	Timer 5
0	13-bit counter/timer				
1	16-bit counter/timer	16-bit timer com auto-reload	16-bit timer com auto-reload	16-bit timer com auto-reload	16-bit timer com auto-reload
2	8-bit counter/timer com auto-reload (Baud rate Generator for UART0 - timer 1)	2x8-bit timer com auto-reload	2x8-bit timer com auto-reload	2x8-bit timer com auto-reload	2x8-bit timer com auto-reload
3	Two 8-bit counter/timer (timer 0)				

Timers 2, 3, 4, e 5

Como configurar?

- Consultar datasheet!!
- Recorrer à ferramenta ConfigWizard2 para ajustes e verificação...

SFR Definition 25.9. TMR2CN: Timer 2 Control

Bit	7	6	5	4	3	2	1	0
Name	TF2H	TF2L	TF2LEN	TF2CEN	T2SPLIT	TR2	T2CSS	T2XCLK
Type	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

SFR Address = 0xC8; SFR Page = 0; Bit-Addressable

7	TF2H	Timer 2 High Byte Overflow Flag. Set by hardware when the Timer 2 high byte overflows from 0xFF to 0x00. In 16 bit mode, this will occur when Timer 2 overflows from 0xFFFF to 0x0000. When the Timer 2 interrupt is enabled, setting this bit causes the CPU to vector to the Timer 2 interrupt service routine. This bit is not automatically cleared by hardware.
2	TR2	Timer 2 Run Control. Timer 2 is enabled by setting this bit to 1. In 8-bit mode, this bit enables/disables TMR2H only; TMR2L is always enabled in split mode.

SFR Definition 25.12. TMR2L: Timer 2 Low Byte

Bit	7	6	5	4	3	2	1	0
Name	TMR2L[7:0]							
Type	R/W							
Reset	0	0	0	0	0	0	0	0

SFR Address = 0xCC; SFR Page = 0

Bit	Name	Function
7:0	TMR2L[7:0]	Timer 2 Low Byte. In 16-bit mode, the TMR2L register contains the low byte of the 16-bit Timer 2. In 8-bit mode, TMR2L contains the 8-bit low byte timer value.

Timers 2, 3, 4, e 5

- Qual o máximo que o Timer 2 consegue contar?
 - 16-bit, 2^{16}
0000h – FFFFh
0 – 65535 => 65536 contagens!
 - Usando um clock de SYSCLK/12, sendo SYSCLK = 48MHz, ocorre um incremento a cada 0.25 us;
 $48 \text{ MHz} / 12 \Rightarrow 4 \text{ MHz} \Rightarrow T = 0.00000025 \text{ s} \Rightarrow 0.25 \text{ us}$
 - $65536 * 0.25 \text{ us} \Rightarrow 1 \text{ overflow a cada } 16.348 \text{ ms} \text{ 😊/😞}$

Timers 2, 3, 4, e 5

- Estratégias para contar tempo (que ultrapasse 16.348ms) ...
 - Usar múltiplos adequados ao problema facilita as operações:
 - 15ms, 10 ms, 5 ms, 1 ms, 100 us...
 - Como contar um segundo (1000 ms)?
 - $0.1 \text{ ms} * 10000 = 1000 \text{ ms}$
 - $1 \text{ ms} * 1000 = 1000 \text{ ms}$
 - $5 \text{ ms} * 200 = 1000 \text{ ms}$
 - $10 \text{ ms} * 100 = 1000 \text{ ms}$
 - $15 \text{ ms} * 60 = 900 \text{ ms}$ e depois contar 100 ms?

16-bit auto Reload Timers: 2, 3, 4, e 5

- **Exemplo – Piscar um led a cada 1 segundo**

1. Programar o Timer 2 em 16-bit autoreload, com overflow a cada 10 ms
 - $10 \text{ ms} * 100 = 1000 \text{ ms!}$
2. Seleccionar o relógio desejado para o timer 2
 - Se utilizarmos $\text{SYSCLK} / 12$, para $\text{SYSCLK} = 48\text{MHz}$, ocorre um incremento (tic) do timer a cada 0,25 us;
3. Escolher o modo de operação do timer 2 – TMR2CN:
 - $\text{T2SPLIT} = 0$ para funcionar como 16 bit auto reload

16-bit auto Reload Timers: 2, 3, 4, e 5

4. Escolher o valor inicial de contagem ascendente e o valor de recarga:

- $10\text{ms} \Rightarrow 10000\text{ us} / 0,25\text{ us} = 40000$ contagens (tics)
- Como é ascendente $\Rightarrow 65536 - 40000 = 25536$ ou -40000

MOV TMR2L, #LOW(25536) ; resto da divisão por 256

MOV TMR2H, #HIGH(25536) ; divisão inteira por 256

TMR2H = (25536) >> 8;

TMR2L = (25536);

TMR2H	TMR2L
FFh	FFh
...	...
...	...
63h	C0h
...	...
...	...
...	...
00h	00h



16-bit auto Reload Timers: 2, 3, 4, e 5

5. Escolher o valor inicial de contagem ascendente e o valor de recarga:
(Como queremos contagens periódicas, a recarga também é para o mesmo valor)

MOV TMR2RL,#LOW(-40000) ;resto da divisão por 256
MOV TMR2RH,#HIGH(-40000) ;divisão inteira por 256

TMR2RH = (25536) >> 8;
TMR2RL = (25536);

6. Colocar o timer a contar:

TR2 = 1;

TMR2H	TMR2L
FFh	FFh
...	...
...	...
63h	C0h
...	...
...	...
...	...
00h	00h



16-bit auto Reload Timers: 2, 3, 4, e 5

```
void timer2_init_auto(int reload){  
  
    TMR2CN = 0;  
    CKCON &= ~(1 << 5);  
  
    TMR2H = (reload) >> 8;  
    TMR2L = (reload);  
  
    TMR2RLH = (reload) >> 8;  
    TMR2RLL = (reload);  
  
}
```

```
timer2_init_auto(-40000);  
TR2 = 1;  
while (!TF2H);  
TF2H = 0;
```

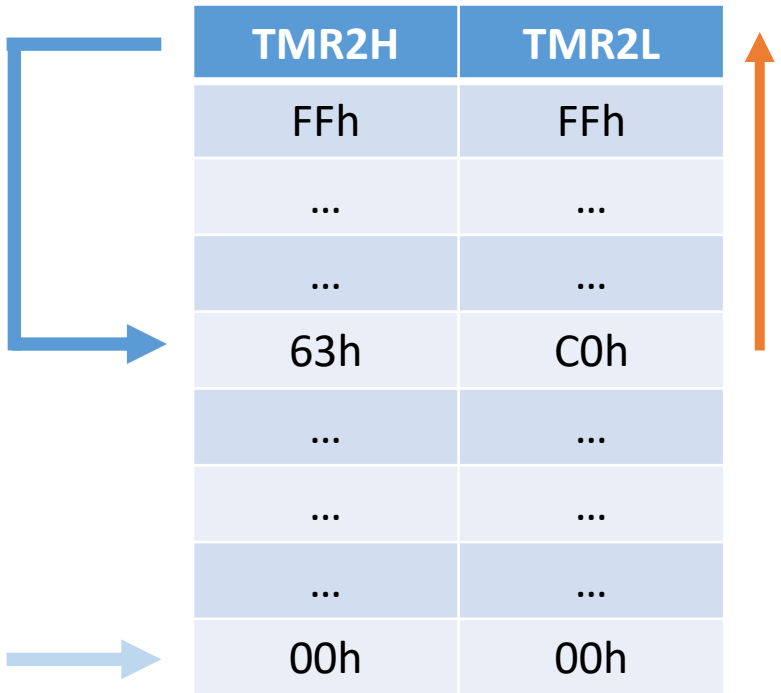
Vantagens e Problemas?

SFR Definition 25.1. CKCON: Clock Control

Bit	7	6	5	4	3	2	1	0
Name	T3MH	T3ML	T2MH	T2ML	T1M	T0M	SCA[1:0]	
Type	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Reset	0	0	0	0	0	0	0	0

Overflow
TF2H = 1
by hardware

Reset by
Software
TF2H = 0;

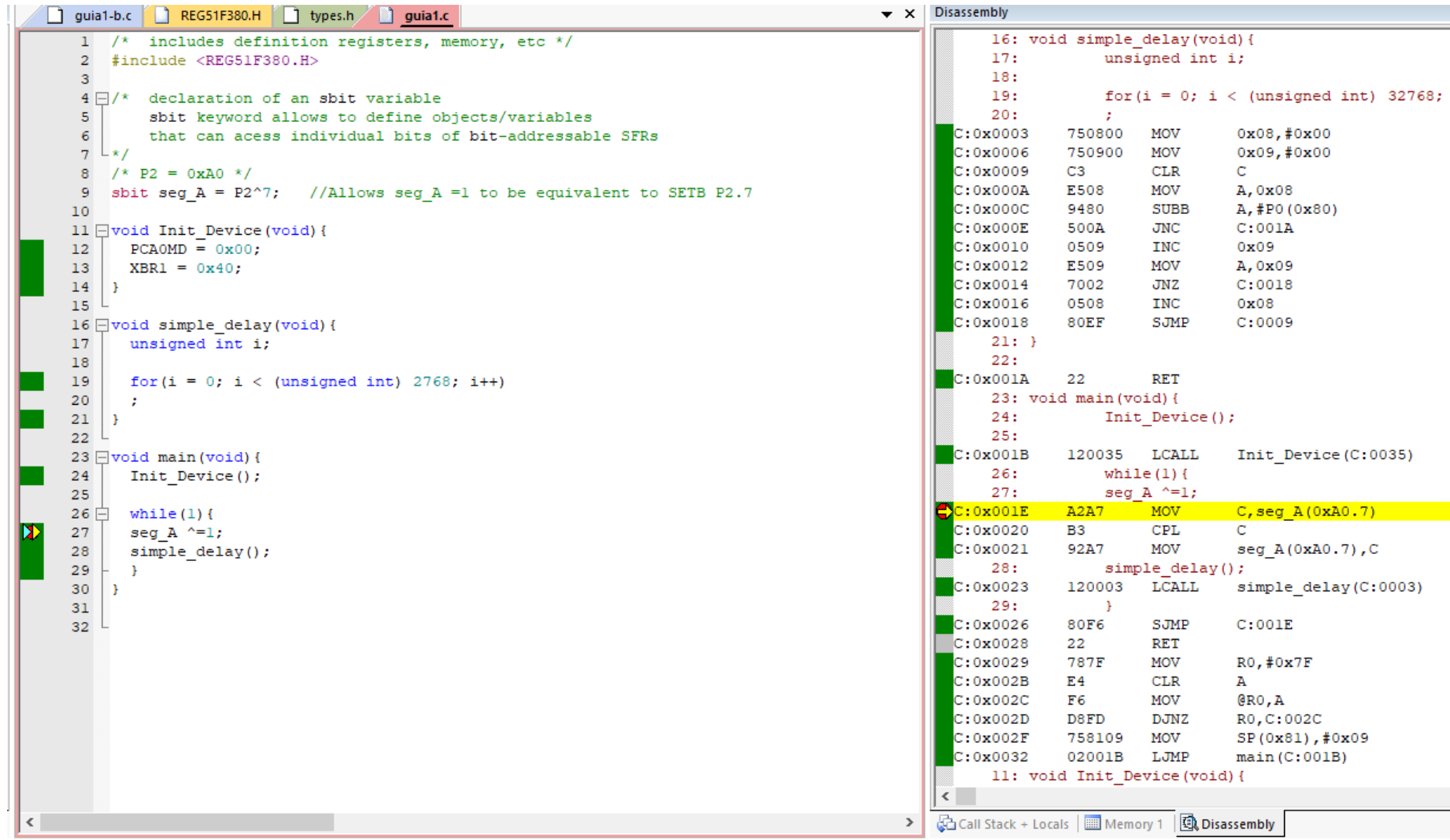


Examplos...

- Testar o exemplo “1-BlinkLed - timer2-polling” ...

Blink Led Example

- Software delay



The screenshot displays an IDE with two panels. The left panel shows the C source code for a blink LED example, and the right panel shows the corresponding assembly disassembly.

C Source Code (gui1.c):

```
1  /* includes definition registers, memory, etc */
2  #include <REG51F380.H>
3
4  /* declaration of an sbit variable
5   sbit keyword allows to define objects/variables
6   that can access individual bits of bit-addressable SFRs
7  */
8  /* P2 = 0xA0 */
9  sbit seg_A = P2^7; //Allows seg_A =1 to be equivalent to SETB P2.7
10
11 void Init_Device(void){
12     PCA0MD = 0x00;
13     XBR1 = 0x40;
14 }
15
16 void simple_delay(void){
17     unsigned int i;
18
19     for(i = 0; i < (unsigned int) 2768; i++)
20     ;
21 }
22
23 void main(void){
24     Init_Device();
25
26     while(1){
27         seg_A ^=1;
28         simple_delay();
29     }
30 }
31
32
```

Assembly Disassembly:

```
16: void simple_delay(void){
17:     unsigned int i;
18:
19:     for(i = 0; i < (unsigned int) 32768; i
20:     ;
C:0x0003 750800 MOV     0x08,#0x00
C:0x0006 750900 MOV     0x09,#0x00
C:0x0009 C3      CLR     C
C:0x000A E508    MOV     A,0x08
C:0x000C 9480    SUBB    A,#P0(0x80)
C:0x000E 500A    JNC     C:001A
C:0x0010 0509    INC     0x09
C:0x0012 E509    MOV     A,0x09
C:0x0014 7002    JNZ     C:0018
C:0x0016 0508    INC     0x08
C:0x0018 80EF    SJMP    C:0009
21: }
22:
C:0x001A 22      RET
23: void main(void){
24:     Init_Device();
25:
C:0x001B 120035 LCALL    Init_Device(C:0035)
26:     while(1){
27:         seg_A ^=1;
C:0x001E A2A7    MOV     C,seg_A(0xA0.7)
C:0x0020 B3      CPL     C
C:0x0021 92A7    MOV     seg_A(0xA0.7),C
28:         simple_delay();
C:0x0023 120003 LCALL    simple_delay(C:0003)
29:     }
C:0x0026 80F6    SJMP    C:001E
C:0x0028 22      RET
C:0x0029 787F    MOV     R0,#0x7F
C:0x002B E4      CLR     A
C:0x002C F6      MOV     @R0,A
C:0x002D D8FD    DJNZ    R0,C:002C
C:0x002F 758109 MOV     SP(0x81),#0x09
C:0x0032 02001B LJMP    main(C:001B)
11: void Init_Device(void){
```

Blink Led Example

- Timer 2 auto-reload (10ms)

```
51 /*****
52  * my main loop
53  *****/
54 void main (void){
55
56     System_Init();
57
58     timer2_init_auto(-40000);
59
60     //Set by hardware when the Timer 2 high byte overflows from 0xFF to 0x00. In 16 bit
61     //mode, this will occur when Timer 2 overflows from 0xFFFF to 0x0000.
62     TF2H = 0;
63     // Enable Flag Timer 2 Overflow
64     ET2 = 1;
65     // Timer 2 Run Control. Timer 2 is enabled by setting this bit to 1.
66     TR2 = 1;
67
68     while(1){
69         while(!TF2H);
70         TF2H = 0;
71         seg_dp ^=1;
72     }
73 }
74
75
76
77
78 }
```

```
10 /*****
11 void System_Init(void){
12     PCA0MD = 0x00;
13     FLSCCL = 0x90;
14     CLKSEL = 0x03;
15
16     XBR1 = 0x40;
17 }
```

E para piscar a cada 1 segundo?

Blink Led Example

- Timer 2 auto-reload (10ms)

```
47 system_init();
48
49 // Set Timer 2 to overflow every 10ms
50 // (with a 4 MHz clock, 10 ms = 40000 ticks)
51 timer2_init_auto(-40000);
52
53 //Set by hardware when the Timer 2 high byte overflows from 0xFF to 0x00. In 16 bit
54 //mode, this will occur when Timer 2 overflows from 0xFFFF to 0x0000.
55 TF2H = 0;
56 // Enable Flag Timer 2 Overflow
57 ET2 = 1;
58
59 // Timer 2 Run Control. Timer 2 is enabled by setting this bit to 1.
60 TR2 = 1;
61
62 while(1){
63
64     while(!TF2H);
65     TF2H = 0;
66
67     // seg_dp ^=1;
68
69     if(--blink == 0){
70         seg_dp ^= 1;
71         blink = 100;
72     }
```

PROBLEMAS?

Interrupts!

Interrupts

- Qualquer evento que interrompe a execução normal do CPU:
 - Externo ou interno ao processador:
 - Exemplos: alteração do valor num pino externo do microcontrolador;
 - Recepção de dados pela porta série;
 - **Overflow de um temporizador.**

Interrupts

- Polling vs. Interrupt?
 - Podemos utilizar o polling para lidar com os eventos;
 - i.e., arrancar um timer e ficar à espera que o evento associado (overflow) ocorra, em teste contínuo;
 - Podemos usar a interrupção:
 - O ciclo do CPU apenas é interrompido quando o evento configurado ocorrer.
 - Liberta “tempo” de execução para outras tarefas.

- ## Vantagens do Polling

- Simples de implementar;
- A execução está sincronizada com a execução do programa, o programador sabe exactamente quando é verificado o estado do dispositivo e quanto tempo será necessário para servi-lo;
- Não será necessário um sistema automático para salvaguardar o estado do processador;
- Não requer atenção à gestão da stack.

- ## Desvantagens do Polling

- Desperdiça tempo de execução, reduzindo por isso tempo disponível para outras actividades;
- Os periféricos não podem ser atendidos de forma instantânea, tornando-se às vezes necessário mais recursos de armazenamento;
- Periféricos menos prioritários podem nunca ser atendidos.

Interrupts

- **Vantagens da Interrupção**

- Não desperdiça tempo de execução na verificação contínua do estado de um periférico;
- O atendimento ao periférico pode ser imediato, fornecendo por isso capacidade de resposta em tempo-real.

- **Desvantagens da Interrupção**

- Requer h/w extra (flags, etc.);
- Introduz um overhead sempre que um periférico interrompe a execução do programa;
- A execução da rotina de serviço à interrupção (ISR) é assíncrona relativamente à execução do programa, dificultando a estimação da temporização da interrupção;
- A gestão de um número excessivo de interrupções pode tornar o sistema lento.

Interrupts

- Interrupções:
 - Sinal a pedir a atenção do processador;
 - Tratamento do sinal por parte do CPU
 - Controlador de Interrupções
- Atendimento da interrupção:
 - **Por software:** rotinas de código, semelhantes às sub-rotinas, que são invocadas automaticamente pelo CPU em resposta ao sinal da interrupção. São designadas por ISR (Interrupt Service Routine).

Interrupts

- Interrupções:
 - Quando surge um sinal de interrupção o CPU não interrompe a instrução actual
 - O sinal é detectado pelo CPU durante o ciclo de execução (fetch-decode-execute) e, após o execute, o CPU pode ignorar ou atender a interrupção (decisão do controlador de interrupções).

Interrupts

- Interrupção vectorizada:
 - Consoante a fonte do sinal, o CPU salta para uma posição da memória de código específica;
- Vector de interrupção:
 - Definidos por hardware;
 - Espaço reservado é pequeno (pode-se colocar um salto para outra posição da memória de código).

Interrupts

- As interrupções provocam uma transferência de execução para um endereço conhecido onde reside uma rotina de serviço à interrupção (ISR);
- O 8051 possui 5 tipos de interrupções mais o RESET:
 - Externas
 - Porta série
 - Timers
 -

Interrupts

Interrupção	Flag que gera Interrupção	Limpa por hardware?	Bit no SFR	Enable Flag	Endereço da ISR
RESET	RST				0000h
Externa 0	IE0	sim	TCON.1	EX0 (IE.0)	0003h
Timer 0	TF0	sim	TCON.5	ET0 (IE.1)	000Bh
Externa 1	IE1	sim	TCON.3	EX1 (IE.2)	0013h
Timer 1	TF1	sim	TCON.7	ET1 (IE.3)	001Bh
Porta Série 0	RI0 ou TI0	não	SCON.0 ou SCON.1	ES0 (IE.4)	0023h
Timer 2	TF2 ou EXF2	não	T2CON.7 ou T2CON.6	ET2 (IE.5)	002Bh

```

CSEG    AT 3h                ; endereço do vector interrupção externa 0
LJMP    ISR_EXT0

CSEG    AT 1000h
ISR_EXT0: PUSH    PSW          ; salvaguarda as flags
          MOV     PSW, #00001000b ; selecciona o banco de registo 1
          ...          ; código da interrupção
          POP     PSW          ; restaura as flags
          RETI
    
```

```

6 ; PLACE CODE AT ADDRESS 0X00
7 CSEG AT 0H
8 LJMP INIT
9
10 ; VECTOR TABLE
11 CSEG AT 50H
12
13 INIT:
14 ; WDT OFF
15 MOV PCA0MD, #0
    
```

Blink Led Example

- Com Interrupção:

```
56  /*****  
57  /* timer 2 isr code here */  
58  void timer2_isr(void) interrupt 5 using 2  
59  {  
60      // sysclock /12 = 4MHz  
61      // 1s 4,000,000 ticks!  
62      // 10ms => 40000 ticks!  
63  
64      if(--blink == 0){  
65          seg_dp ^= 1;  
66          blink = 100;  
67      }  
68  
69      TF2H = 0;  
70  }
```

```
74  void main (void){  
75  
76      System_Init();  
77  
78      timer2_init_auto(-40000);  
79  
80      //Set by hardware when the Timer 2 high byte overflows from 0xFF to 0x00. In 16 bit  
81      //mode, this will occur when Timer 2 overflows from 0xFFFF to 0x0000.  
82      TF2H = 0;  
83      // Enable Flag Timer 2 Overflow  
84      ET2 = 1;  
85      // Timer 2 Run Control.  
86      // Timer 2 is enabled by setting this bit to 1.  
87      TR2 = 1;  
88      // Enable All Interrupts.  
89      EA = 1;  
90  
91      while(1);  
92      // do whatever you want!!!!!!!!!!!!  
93  
94  }
```



Examplos...

- Testar o exemplo “2-BlinkLed - timer2-interrupt”...

Software timers

Software timers

Como controlar de forma independente diferentes contagens usando o mesmo hardware timer?

- Implementando software timers...
 - Multiplexar um hardware timer, e.g., timer 2
 - Escolher a base de contagem do timer apropriada (*timer tick*)
 - Configurar o timer para funcionar com a interrupção no ***tick*** escolhido
 - Escolher o número de temporizadores virtuais apropriado
 - Deixando tempo/ciclos suficientes para executar o restante código

Software timers

- **Usando Timer 2:**
 - 16 bit auto reload
 - Clock source SYSCLK / 12
 - Timer tick = 1ms
 - configurar a interrupção para ocorrer a cada 1ms
 - Base de tempo passa a ser 1ms
 - Pode ser usada para controlar vários software timers que funcionem dentro desta resolução...
 - Permite controlar eventos com diferentes requisitos temporais...
 - 100ms, 500 ms, 231 ms, 3 ms, 3450ms, etc.

Software timers

- Para 4 software timers com tick 1ms...

```
// Set 4 software timers
#define NUM_SOFT_TIMERS 4

// Set timer tick to 1 ms
#define TIMER_TICK_MS 1

// Initiate software timers array
unsigned int soft_timers[NUM_SOFT_TIMERS] = {0};
```

- Init:

```
/**
 * Set a software timer
 */
void set_software_timer(unsigned char timer_id, unsigned int ms_duration) {
    if (timer_id < NUM_SOFT_TIMERS) {
        soft_timers[timer_id] = ms_duration / TIMER_TICK_MS; // Convert ms to tick count
    }
}
```

Software timers

- Modificando a ISR do Timer 2 (função de init mantém-se)
 - A cada interrupção (1 ms) decrementa-se cada soft timer até expirar (= 0)

```
/**
 * Timer2 ISR
 */
void timer2_isr(void) interrupt 5 using 2 {
    unsigned char i;

    // Decrement each software timer if it's not zero
    for (i = 0; i < NUM_SOFT_TIMERS; i++) {
        if (soft_timers[i] > 0) {
            soft_timers[i]--;
        }
    }

    TF2H = 0; // Clear Timer 2 overflow flag
}
```

Software timers

- Criar um software timer:
 - Deve ser chamado sempre que o software timer expira...

```
// Set multiple software timers with different intervals
set_software_timer(0, 1000); // 1 second timer
set_software_timer(1, 2000); // 2 second timer
set_software_timer(2, 500);  // 500ms timer
set_software_timer(3, 3000); // 3 second timer
```

Software timers

- No main loop verificar quando o software timer expirou:
 - Fazer a acção e fazer novamente set ao timer correspondente

```
while (1) {  
    // Check if any software timer has expired  
    if (soft_timers[0] == 0) {  
        seg_A ^= 1; // Toggle segA  
        set_software_timer(0, 1000); // Reset to 1 second  
    }  
    if (soft_timers[1] == 0) {  
        seg_B ^= 1; // Toggle segB  
        set_software_timer(1, 2000); // Reset to 2 seconds  
    }  
    if (soft_timers[2] == 0) {  
        seg_C ^= 1; // Toggle segC  
        set_software_timer(2, 500); // Reset to 500ms  
    }  
    if (soft_timers[3] == 0) {  
        seg_D ^= 1; // Toggle segD  
        set_software_timer(3, 3000); // Reset to 3 seconds  
    }  
}
```

Exemplos...

- Testar o exemplo “3-BlinkLed - timer2-interrupt-swtimers-noCB”

Software timers

- Melhorando o exemplo anterior...
 - Criar funções de callback para cada software timer
 - Funções que são chamadas dentro da ISR
 - Devem conter o minimo de codigo possivel
 - Fazem acções curtas, ou activam flags que podem ser usadas no main loop

```
// Set 4 software timers
#define NUM_SOFT_TIMERS 4
// Set timer tick to 1 ms
#define TIMER_TICK_MS 1

// defines a new type, timer_callback_t, for function pointer
typedef void (*timer_callback_t)(void);

// Initiate software timers array
unsigned int soft_timers[NUM_SOFT_TIMERS] = {0};

// Array of callback functions
timer_callback_t callbacks[NUM_SOFT_TIMERS];
```

Software timers

- Definir a função de callback

```
/* *****  
 * Example callback functions *  
***** */  
void timer0_callback(void) {  
    seg_A ^= 1; // Toggle segA  
    set_software_timer(0, 250, timer0_callback);  
}
```

- Modificar a função de criar o software timer

```
/* *****  
 * Set a software timer with a callback *  
***** */  
void set_software_timer(unsigned char timer_id, unsigned int ms_duration, timer_callback_t callback)  
{  
  
    if (timer_id < NUM_SOFT_TIMERS) {  
        soft_timers[timer_id] = ms_duration / TIMER_TICK_MS; // Convert ms to tick count  
        callbacks[timer_id] = callback; // Assign the callback function  
    }  
}
```

Software timers

- Modificar a função da ISR para chamar a funções de callback

```
/******  
* Timer 2 ISR *  
*****/  
void timer2_isr(void) interrupt 5 using 2 {  
    unsigned char i;  
  
    // Decrement each software timer if it's not zero  
    for (i = 0; i < NUM_SOFT_TIMERS; i++) {  
        if (soft_timers[i] > 0) {  
            soft_timers[i]--;  
  
            // Check if the timer has reached zero and a callback is set  
            if (soft_timers[i] == 0 && callbacks[i] != 0) {  
                // Call the associated callback function  
                callbacks[i]();  
            }  
        }  
    }  
  
    TF2H = 0; // Clear Timer 2 overflow flag  
}
```

Software timers

- Criar um software timer com a função de call back correspondente

```
// Set multiple software timers with different intervals and callbacks
set_software_timer(0, 250, timer0_callback); // 250 ms, callback on expiration
set_software_timer(1, 300, timer1_callback); // 300 ms
set_software_timer(2, 621, timer2_callback); // 621 ms
set_software_timer(3, 78, timer3_callback); // 78 seconds
```

- O main loop...

```
while (1) {

    // Do what ever you want HERE!!

}
```

THANK YOU!

Questions?

mr.gomes@dei.uminho.pt